

# Вайб-коддинг: от курса к будущему

Финальная лекция

1.0

2.0

3.0



# Путь, который мы прошли

От классического программирования через автоматизацию к AI-ассистируемой разработке



# Три эпохи программирования



Software 1.0



Software 2.0



Software 3.0



1.0: Классический код



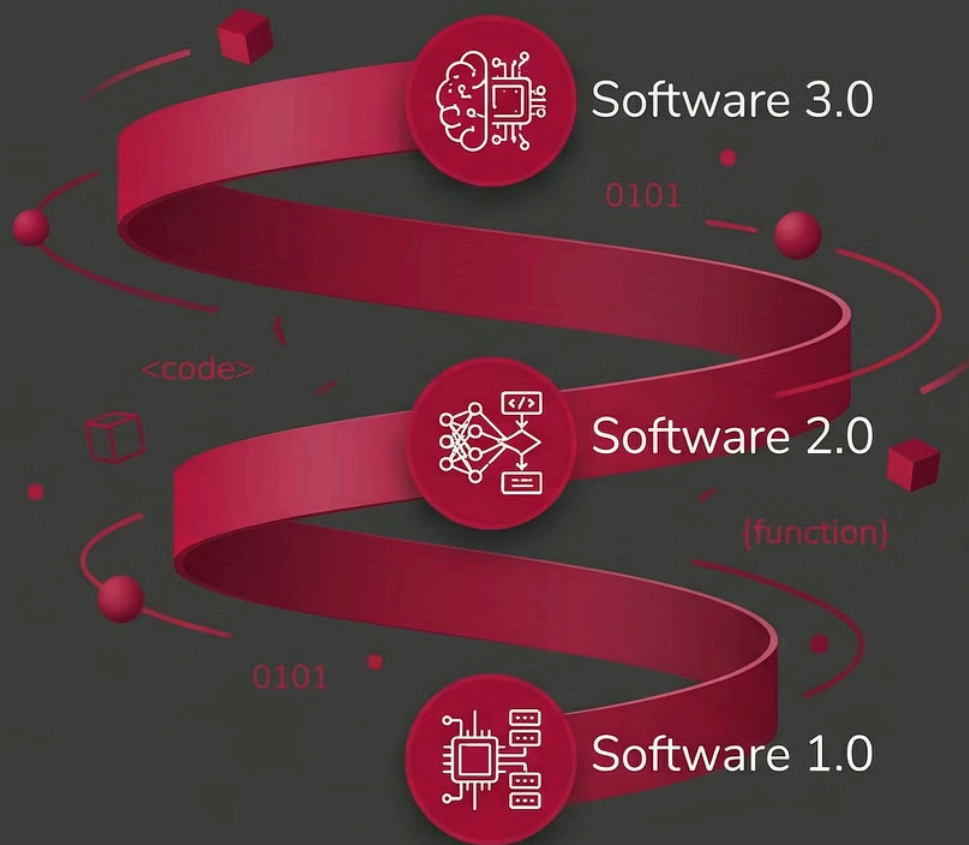
2.0: Машинное обучение



3.0: Синергия человека и AI

# 1. Архитектура вашего обучения

## Спираль знаний



# Ваша позиция на спирали эволюции



1. Понимать классические принципы



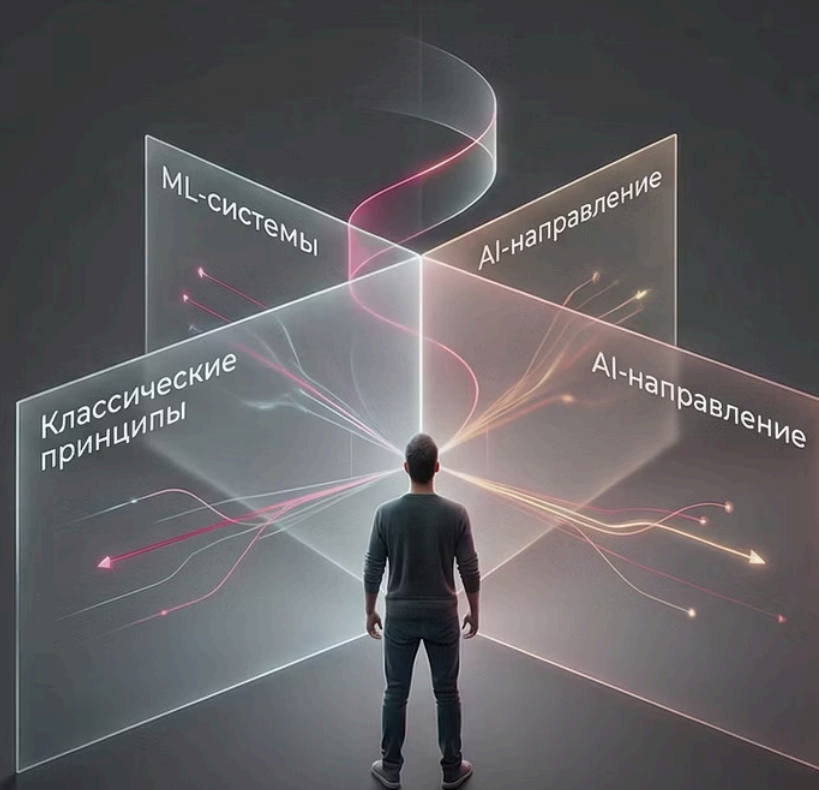
2. Знать ML-системы



3. Уметь направлять AI



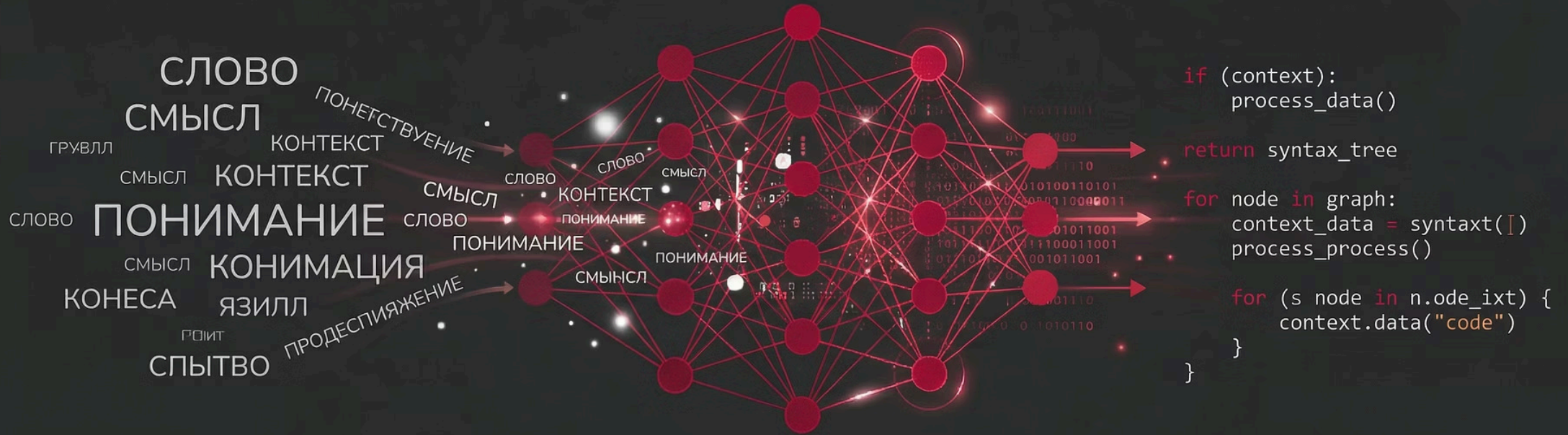
4. Критически мыслить во всех трёх



# Git как метафора: Версионирование мышления



# LLM: Новое понимание языка



Паттерны, не понимание



Естественный язык → Синтаксически корректный код

# Критическое понимание LLM



Галлюцинации



Предвзятость данных



Отсутствие реального понимания

**Понимание ограничений — необходимое условие  
ответственного использования**

# AI-ассистенты: От инструмента к партнёру



Они расширяют вашу пропускную способность

# Правильное использование AI-ассистентов



**1.**

**Ясное** формулирование намерения



**2.**

**Проверка** output перед применением

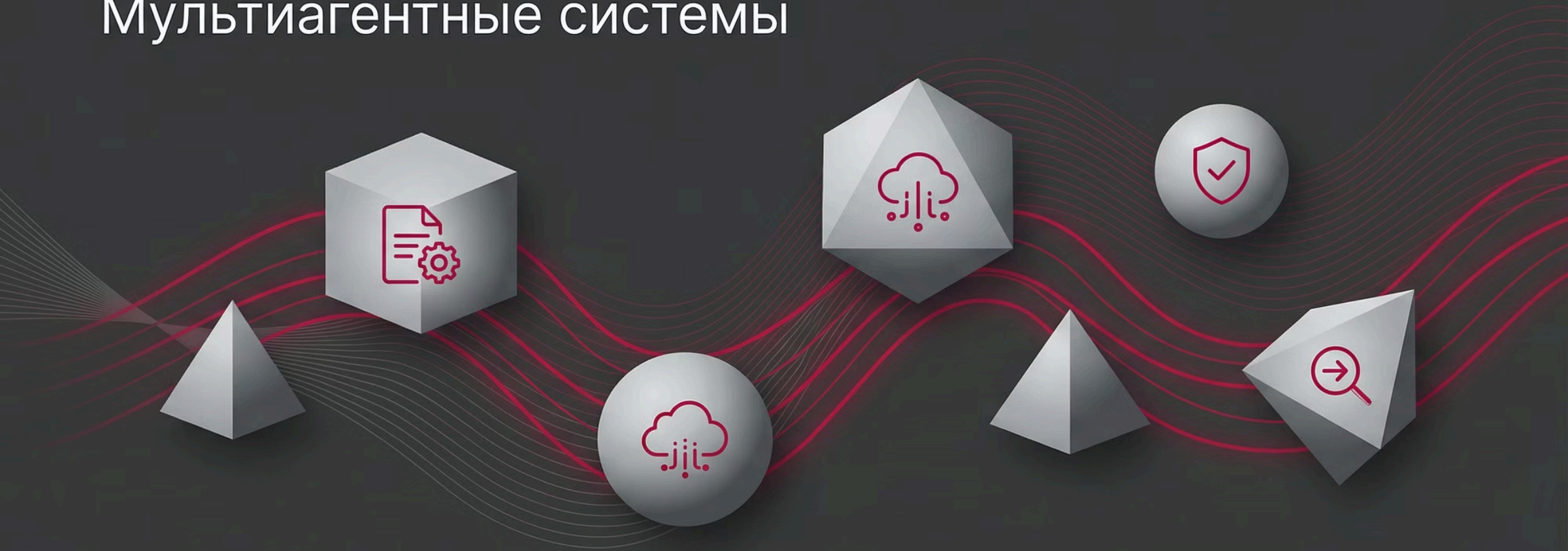


**3.**

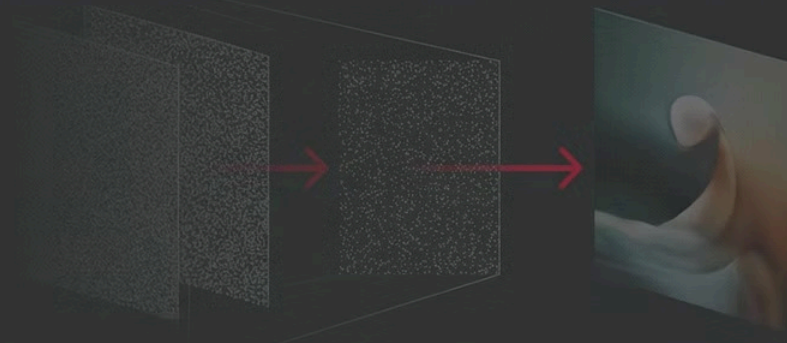
**Ответственность** за качество

# 2. Полифония инструментов

Мультиагентные системы



# Gamma как пример: Симфония агентов



**1. Text Encoder**  
(understands  
description)



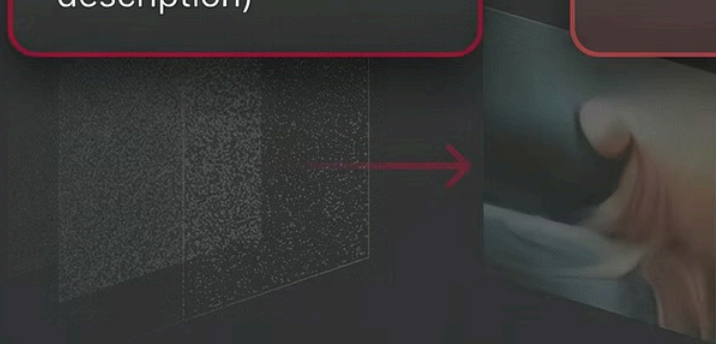
**2. Diffusion Agent**  
(generates iteratively)



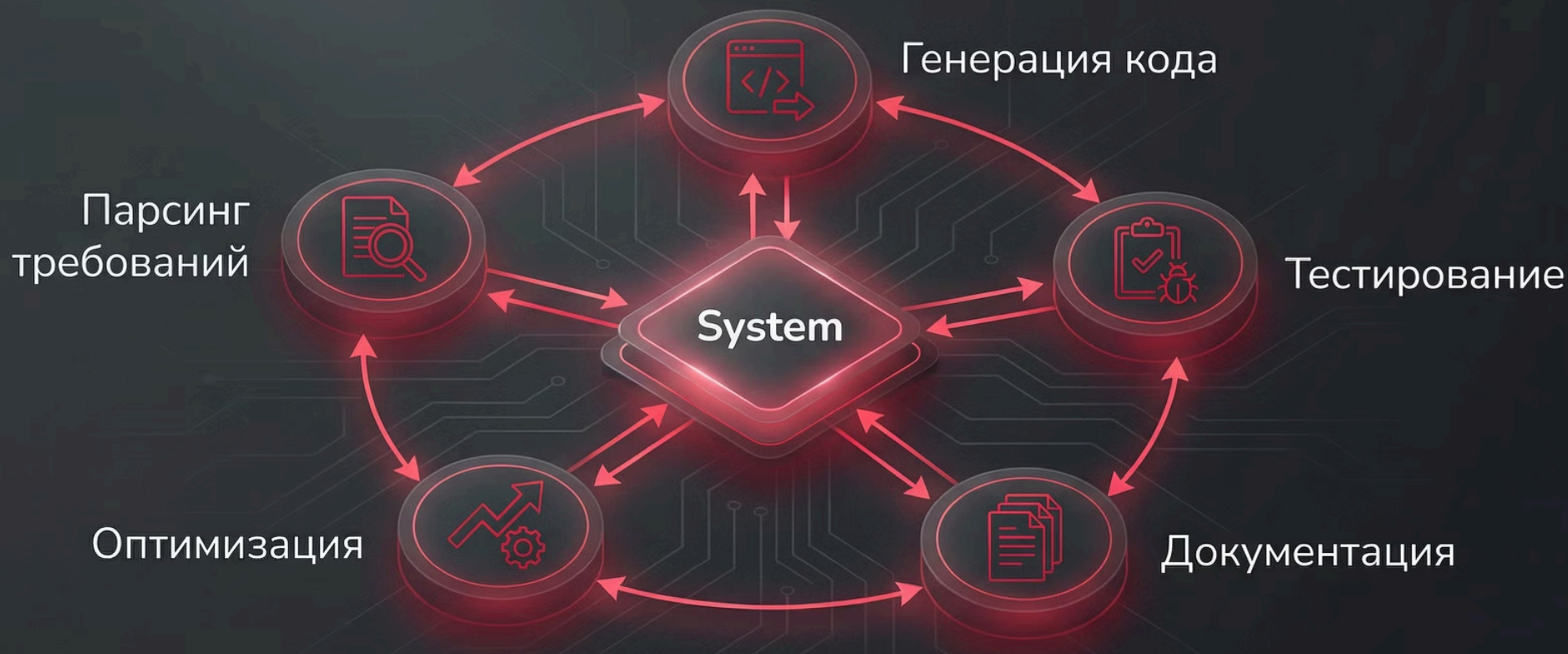
**3. Quality Agent**  
(evaluates)



**4. Refinement Agent**  
(improves)



# Агенты в реальной разработке

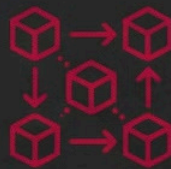
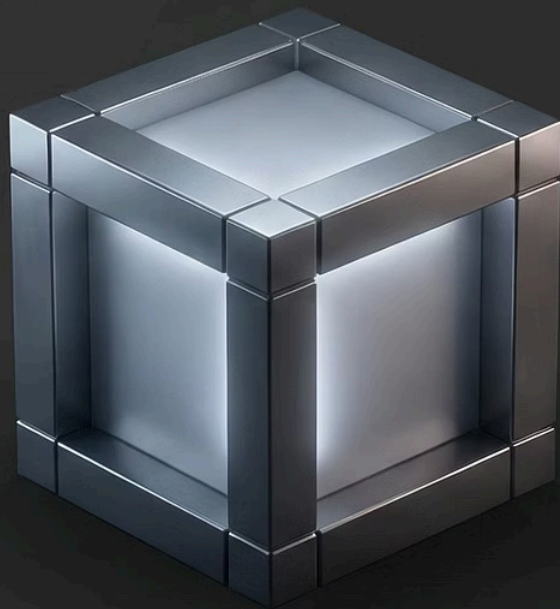


Экосистема специализированных компонентов

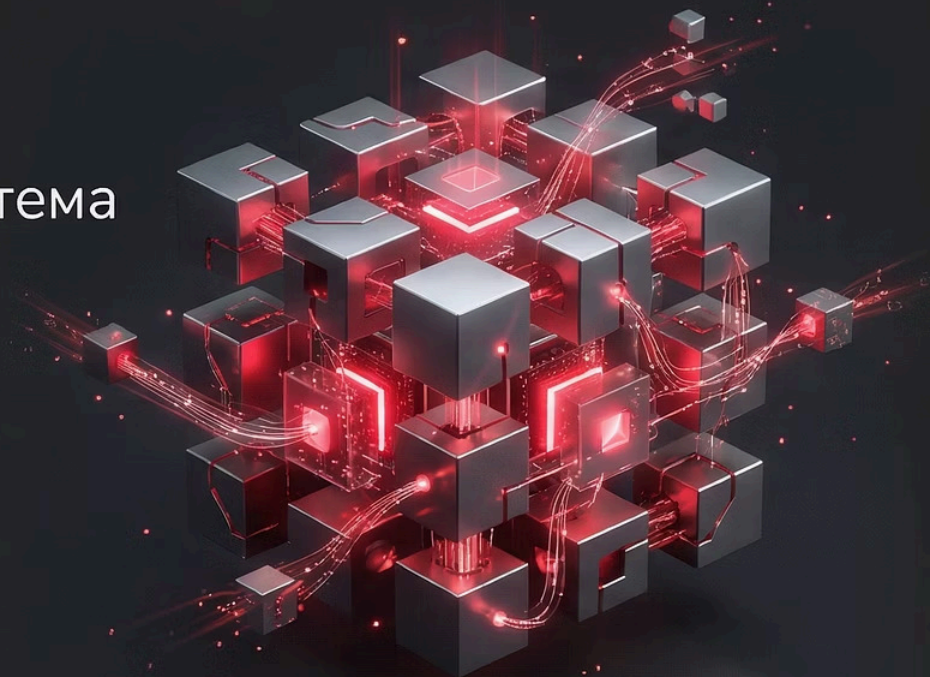
# Навык будущего: Видеть сложные системы как экосистему специализированных компонентов



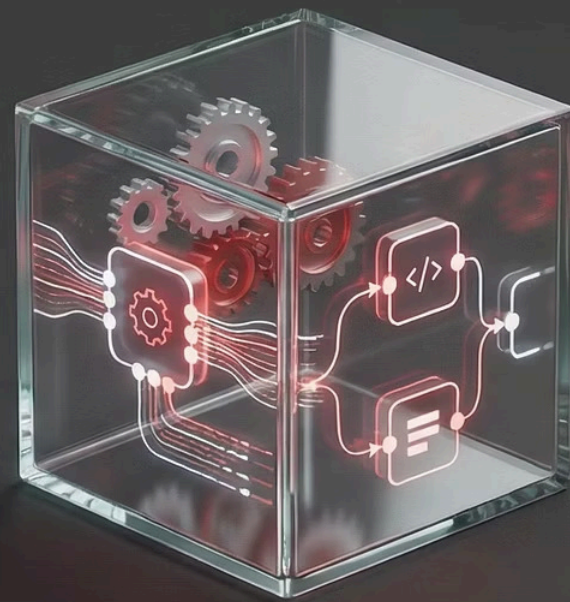
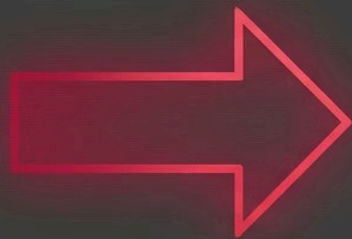
Монолит



Экосистема



# Визуальная отладка: От чёрного ящика к стеклянному



Явный поток  
данных



Локализуемые  
ошибки



Инженерная  
оптимизация

# 3. Политика вайб-кодинга

Ответственность в эпоху AI



# Этика AI-сгенерированного кода

Скорость приносит обязательства

Скорость  + AI код `[</>]` = Ответственность 



Собственность  
и лицензирование



Безопасность  
и надёжность



Предвзятость и  
справедливость

# Собственность и лицензирование



1. Код может  
содержать генетику  
открытых данных



2. Вы ответственны  
за проверку  
лицензий



3. Используйте  
функцию проверки  
источника



GitHub Copilot может указать источник — **ИСПОЛЬЗУЙТЕ ЭТУ ФУНКЦИЮ**

# Безопасность и надёжность



Аутентификация



Работа  
с деньгами



Персональные  
данные



Тесты — это  
**МИНИМУМ**

# Предвзятость и справедливость



**ВАША РОЛЬ** — видеть эти  
паттерны и **противостоять** им



# От «ассерпт all» к ответственному использованию

Начинающий подход

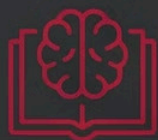
АССЕРПТ ALL



Зрелое использование



Принципы ответственному использованию



Понимание  
принимаемого



Осознание  
рисков



Ответственность перед  
пользователями

# 4. Практические навыки для будущего

Будущие навыки введённого будущего важно не упустить из виду, что вопреки тому, так технология продвигается в различных паттернах и регистрирует существенные навыки. В настоящее время это отражение прогресса и инноваций.



# Hard Skills: Техническое ядро



1. Prompt Engineering



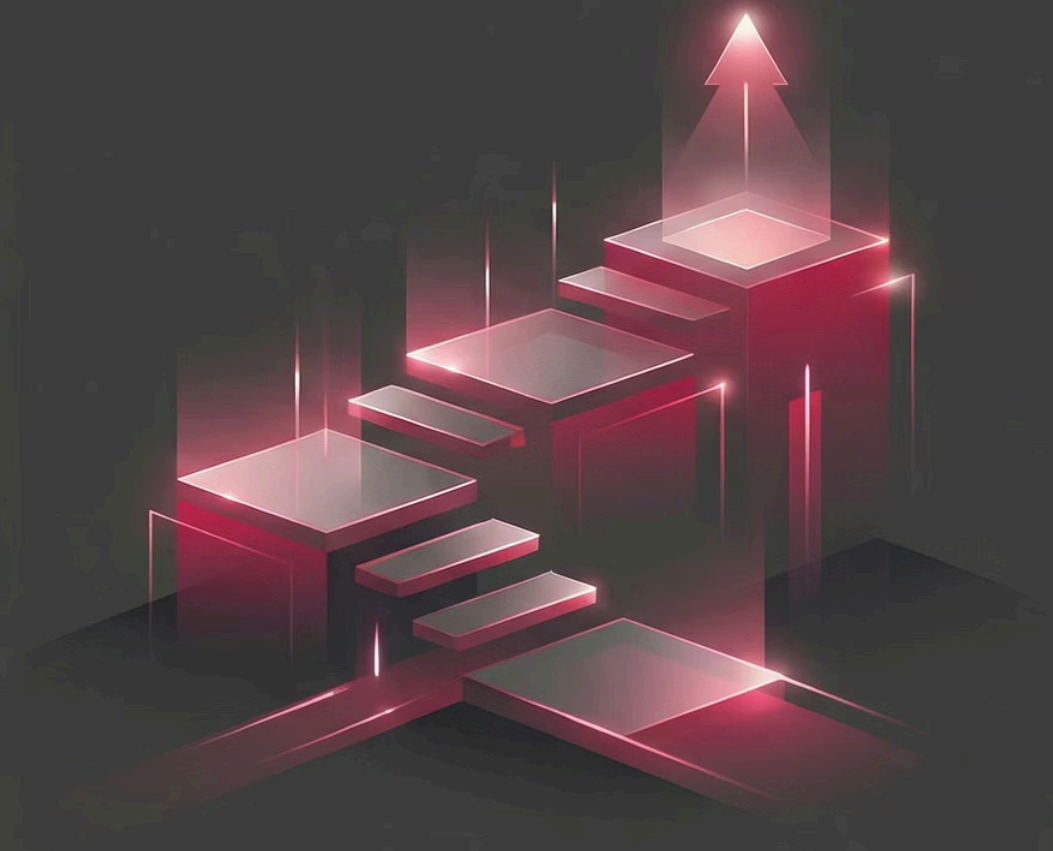
2. Code Review в эру AI



3. Мультиагентные системы



4. Непрерывное обучение

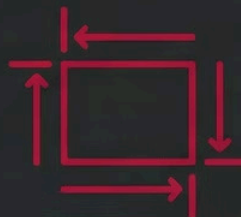


# Prompt Engineering — это инженерия, не магия

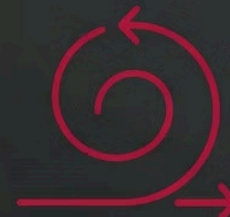
Специфичность  
задачи



Контекст и  
ограничения



Итеративное  
уточнение



Сгенерируй картинку с котом



Создай фотореалистичное изображение рыжего кота в профиль, сидящего на подоконнике при естественном дневном свете, с детальной текстурой шерсти и мягким фокусом на саду за окном. Фотография.

Rough Prompt

Precise Prompt

# Code Review в эру AI



**Не синтаксис**



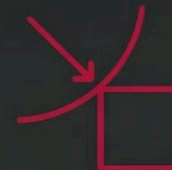
**Смысл и паттерны**



Проверять смысл,  
не синтаксис

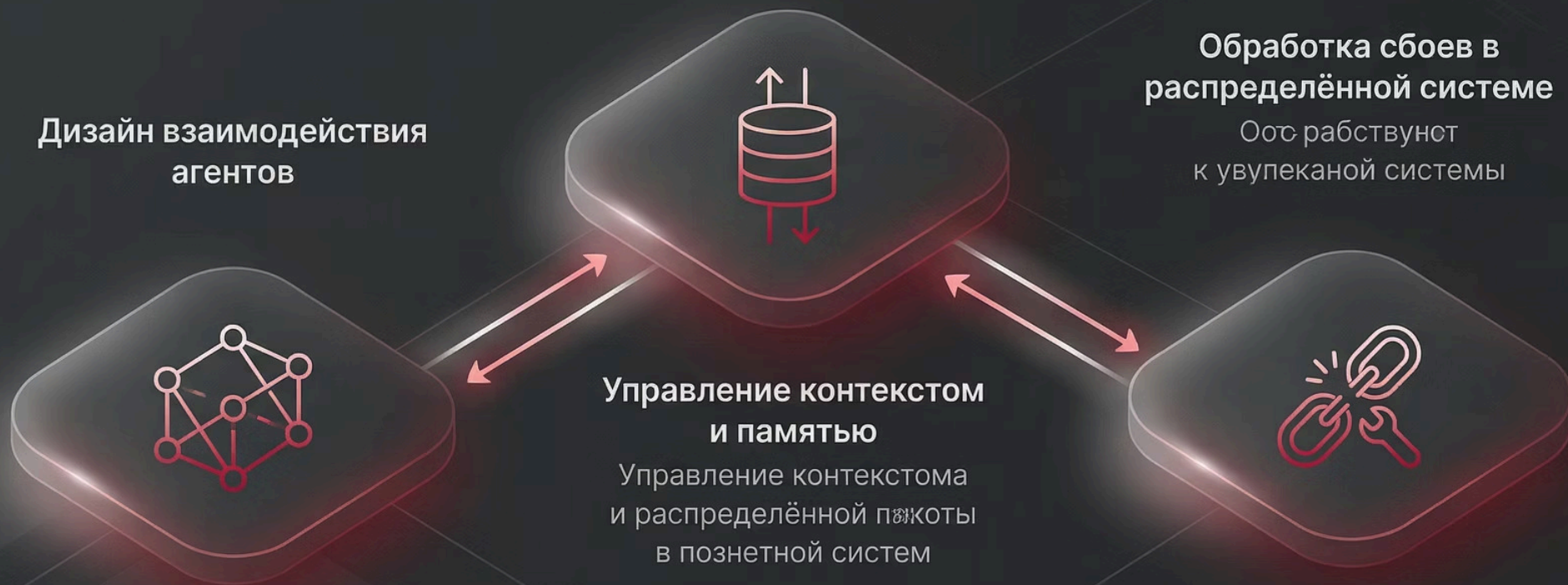


Видеть нарушения  
паттернов



Тестировать  
граничные случаи

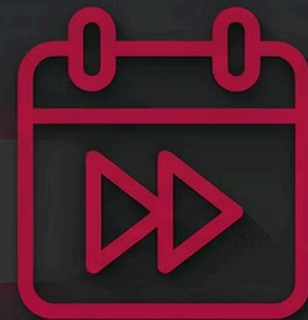
# Мультиагентные системы



# Непрерывное обучение в AI-сфере



Привычка экспериментировать — это инвестиция



# Soft Skills: Человеческое ядро

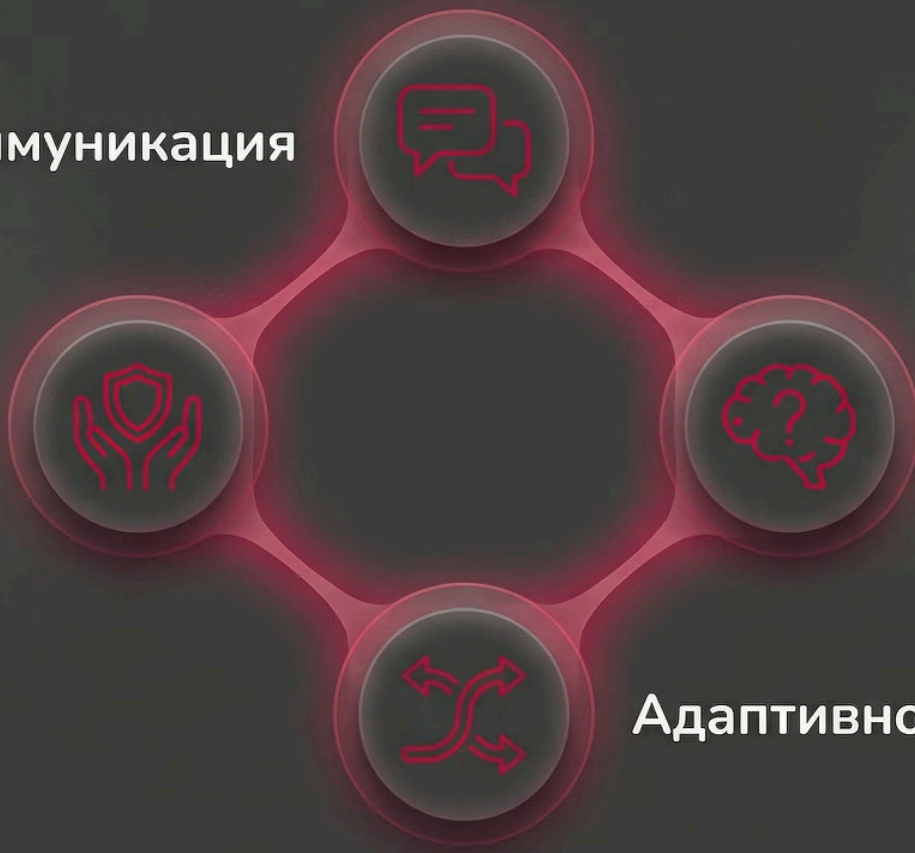
В эпоху AI — люди остаются людьми

Ответственность

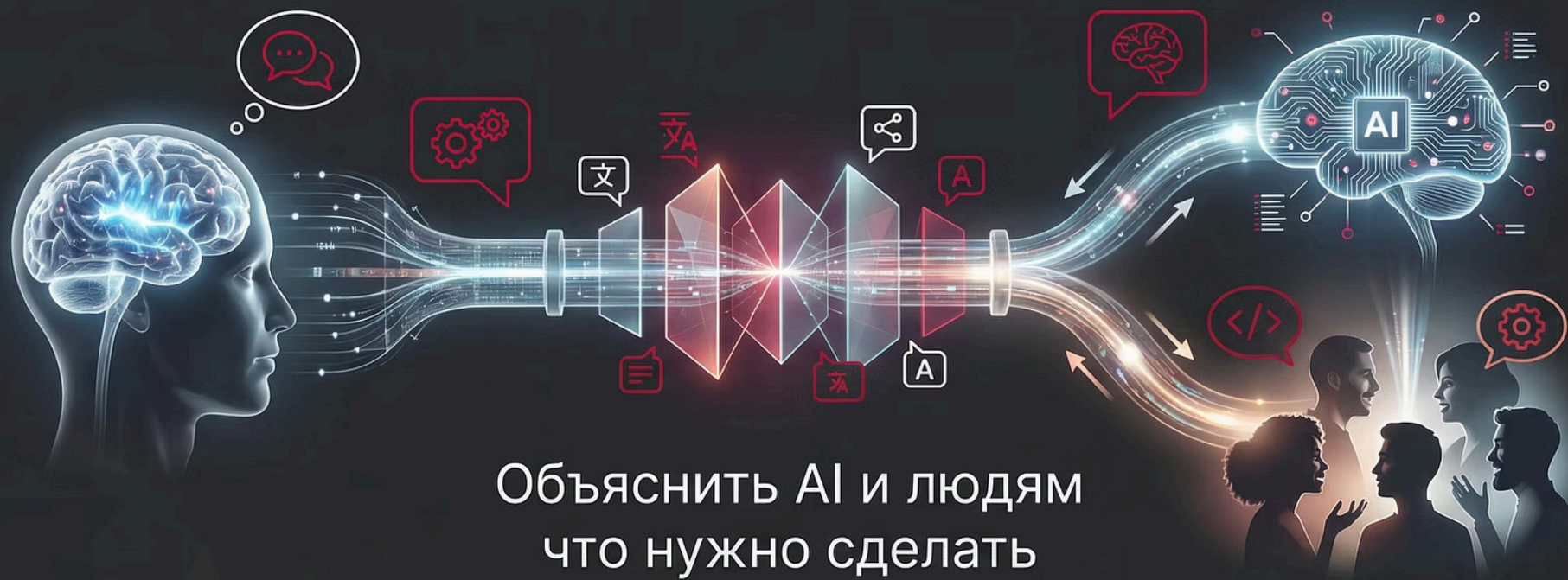
Коммуникация

Критическое  
мышление

Адаптивность



# Коммуникация



Объяснить AI и людям  
что нужно сделать

Ясность мышления → ясность кода

# Критическое мышление



**ПРОВЕРЕННОЕ  
РЕШЕНИЕ**



Не принимать  
на веру



Вопрос: почему  
это решение?



Видеть что компьютер  
не может видеть

# Адаптивность



Платформы  
эволюционируют



Инструменты  
меняются  
каждый месяц



Платформы  
эволюционируют



Ваша гибкость —  
конкурентное  
преимущество

Январь

Февраль

Март

Апрель

Май

Июнь

# Ответственность

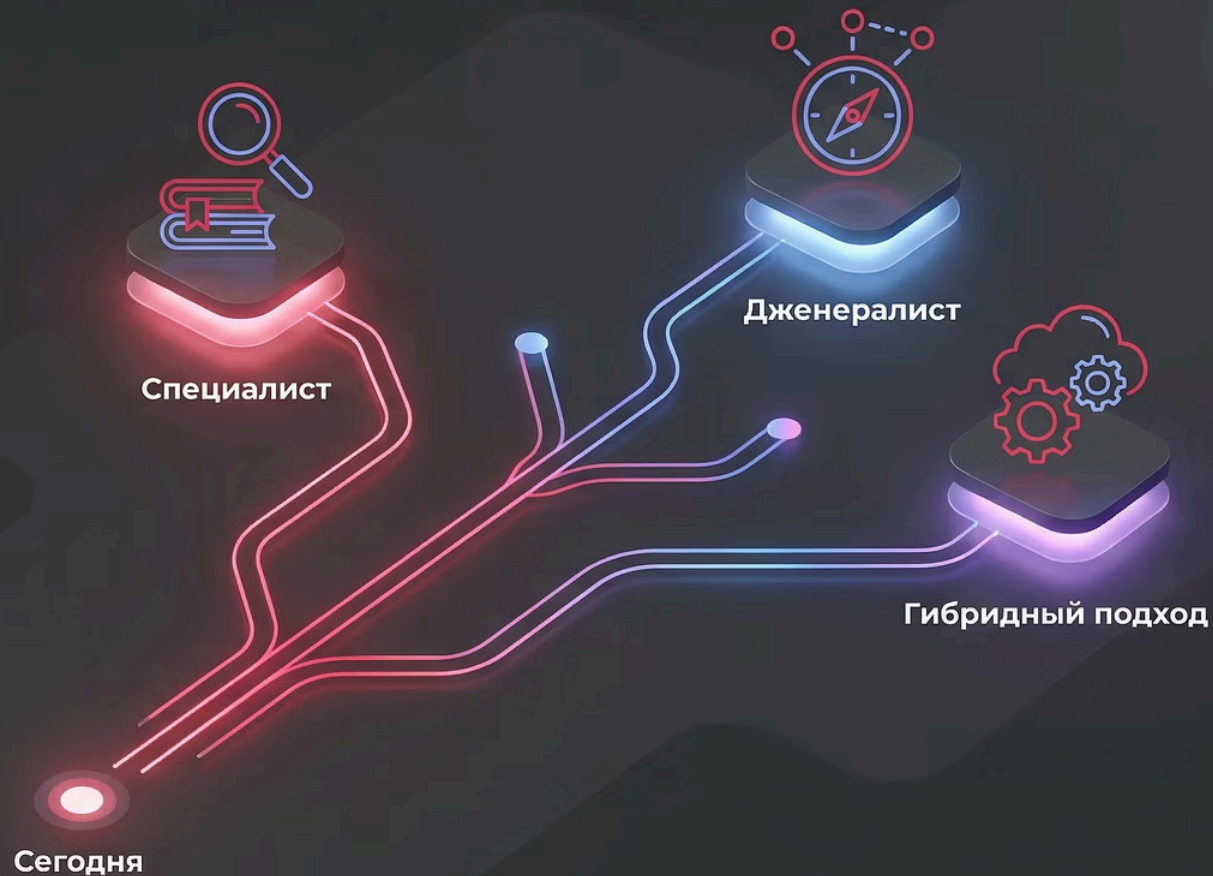


**Ошибка в AI-генерированном коде — ваша ошибка**

**Этика не опция**

# 5. Карьерные направления

От курса к профессии



# Специалист vs Генералист

## Специалист (Deer)



Глубокое знание

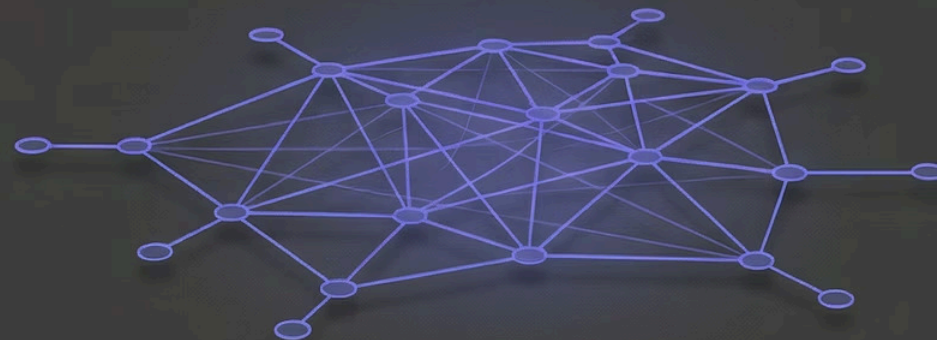


Высокая зарплата



Узкие возможности

## Генералист (Broad)



Компетентность во многом



Больше возможностей



Сложнее конкурировать

# Гибридный подход

## Глубина в 1-2 областях

Глубина профессиона яввляется также интенсивности, анализа, на воды на опноенной. жижи оеденка ютие тиюгю трафонсоканнк и оплаюнальнее-профессиока модел.



## Базовое понимание остального

Базовое понимание остального, для пониманство с наюпрофессноннам, глубина и стратеги, баланса и оптималь.

Гибкость + Экспертиза  
одновременно

# Где работает вайб-кодинг лучше всего?

1. ⚡ Прототипирование и MVP
2. 📄 Boilerplate и шаблоны
3. 📋 Тесты и документация
4. 🔄 Рефакторинг
5. 🧪 Экспериментирование



# Где нужна осторожность?



## 1. Критические алгоритмы

Преподлагает вде подпонино  
намовниемы критически логинов.



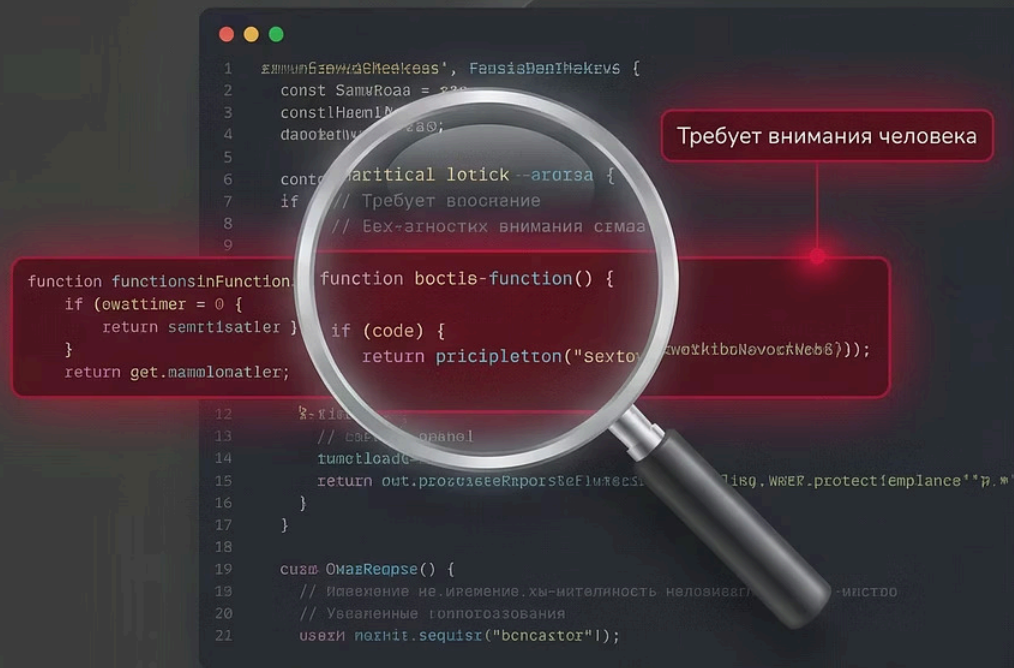
## 2. Безопасность



## 3. Производительность



## 4. Архитектура



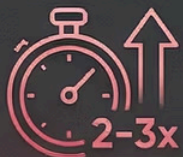
# Три трека развития



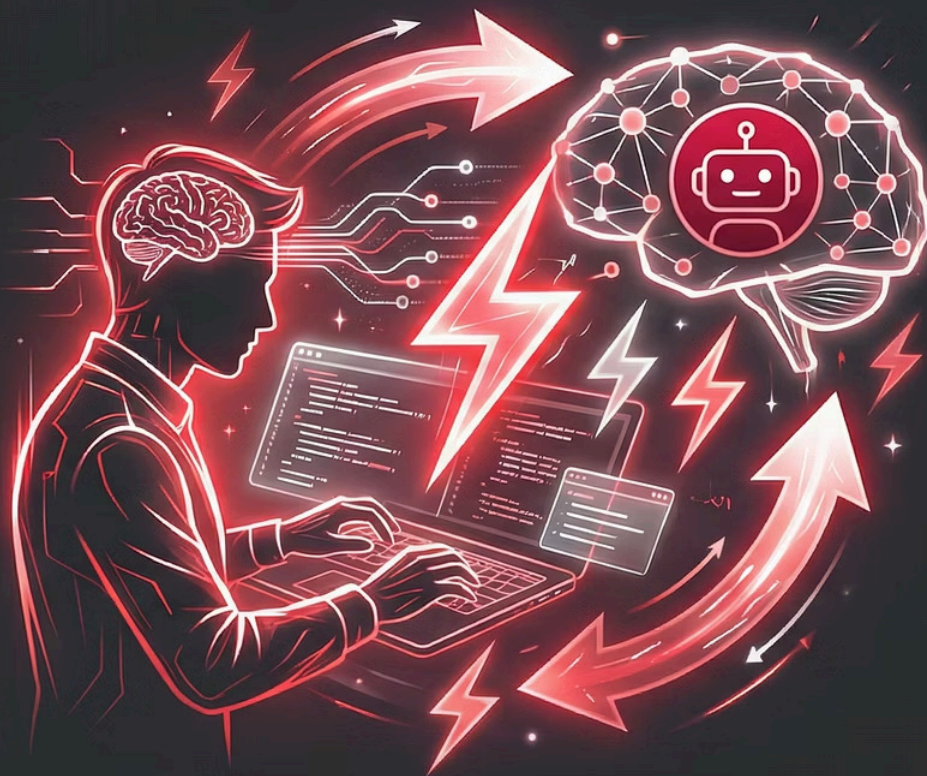
# Трек 1: Full-stack разработчик + AI



Master  
вайб-кодинг



Разработка  
в 2-3х быстрее



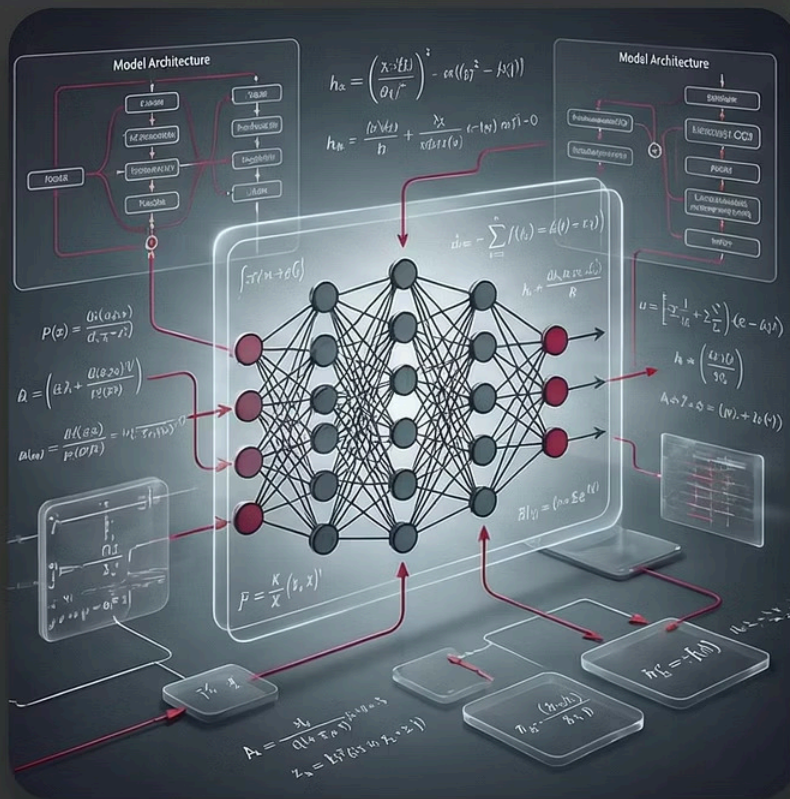
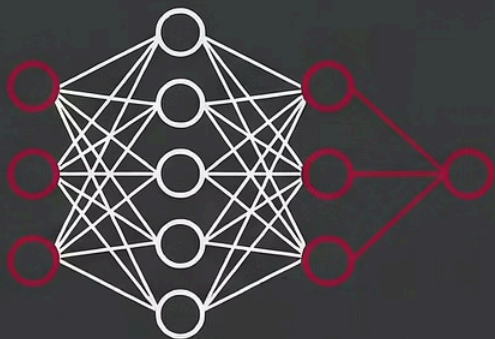
Карьера:  
стартапы,  
MVP-роли



Зарплата:  
выше среднего

# Трек 2: AI/ML специалист

- Глубокое понимание моделей
- Не использование, но создание AI-систем



Карьера:  
ML Engineer,  
AI Researcher,  
LLMOps

Зарплата:  
топ-уровень

# Трек 3: Engineering Leader/Architect



Стратегическое  
видение

Управление AI-  
ассистируемыми  
командами



Карьера: Tech  
Lead, Architect,  
Engineering  
Manager

Зарплата:  
высокий  
потенциал

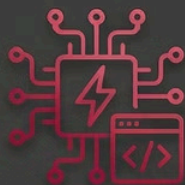
# Как выбрать свой путь?



1. Я предпочитаю быстро доставлять или глубоко понимать?



2. Хочу быть исполнителем или архитектором?



3. Энергия в коде или в людях?



4. Ищу стабильность или адреналин?



# 6. Практический план на 6 месяцев



Месяцы 1-2



Месяцы 3-4



Месяцы 5-6

# Месяцы 1-2: Закрепление



3-5 реальных  
проектов



Анализ: где AI  
помог, где  
ошибся



Ведите дневник  
рефлексии



2-3 статьи  
об этике AI



# Месяцы 3-4: Углубление

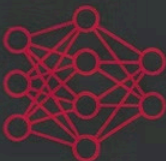
Выберите ОДИН из путей



## Путь А: Full-stack

**Описание:** Разработка и интеграция всех уровней системы.

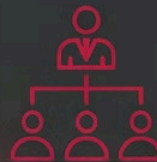
**Результат:** Полностью функциональное приложение.



## Путь В: ML

**Описание:** Создание, обучение и внедрение моделей машинного обучения.

**Результат:** Предиктивные возможности и анализ данных.



## Путь С: Leadership

**Описание:** Управление командой, планирование и стратегия.

**Результат:** Успешная координация и достижение целей.

# Путь А: Full-stack



## 1 Изучите мультиагентную систему

(Используйте n8n или LangChain для интеграции)



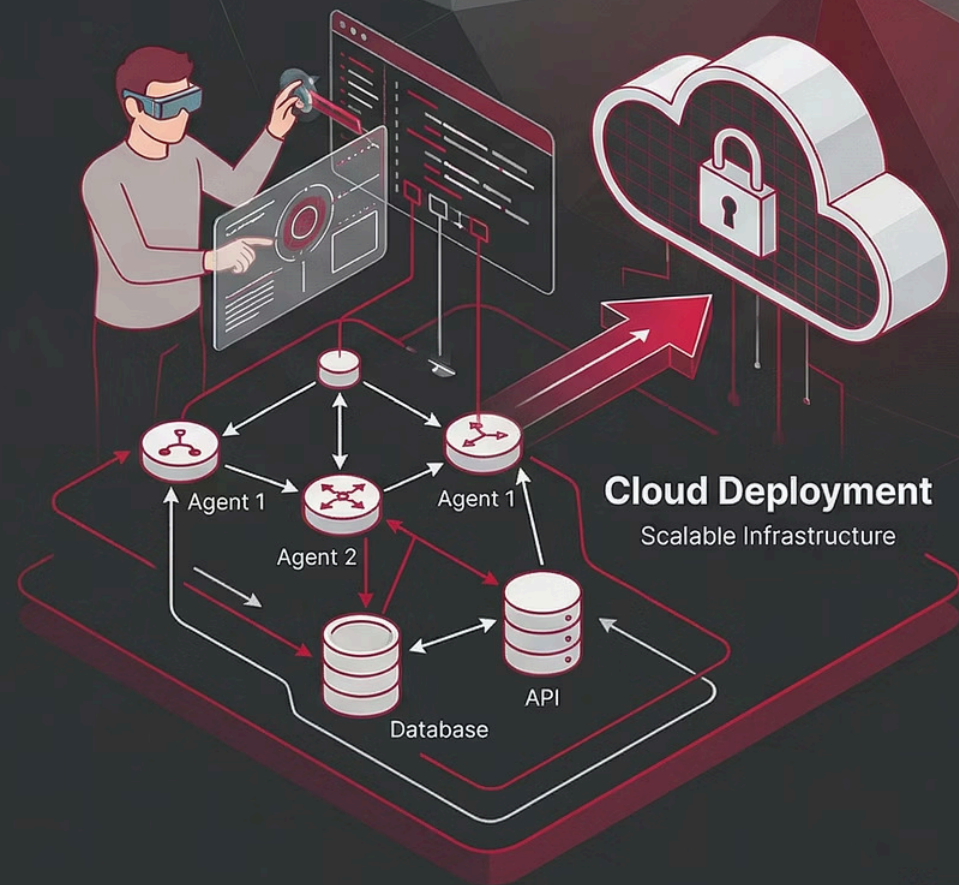
## 2 Создайте свой AI-ассистент

Разработайте персонализированный, обучаемый ИИ для задач



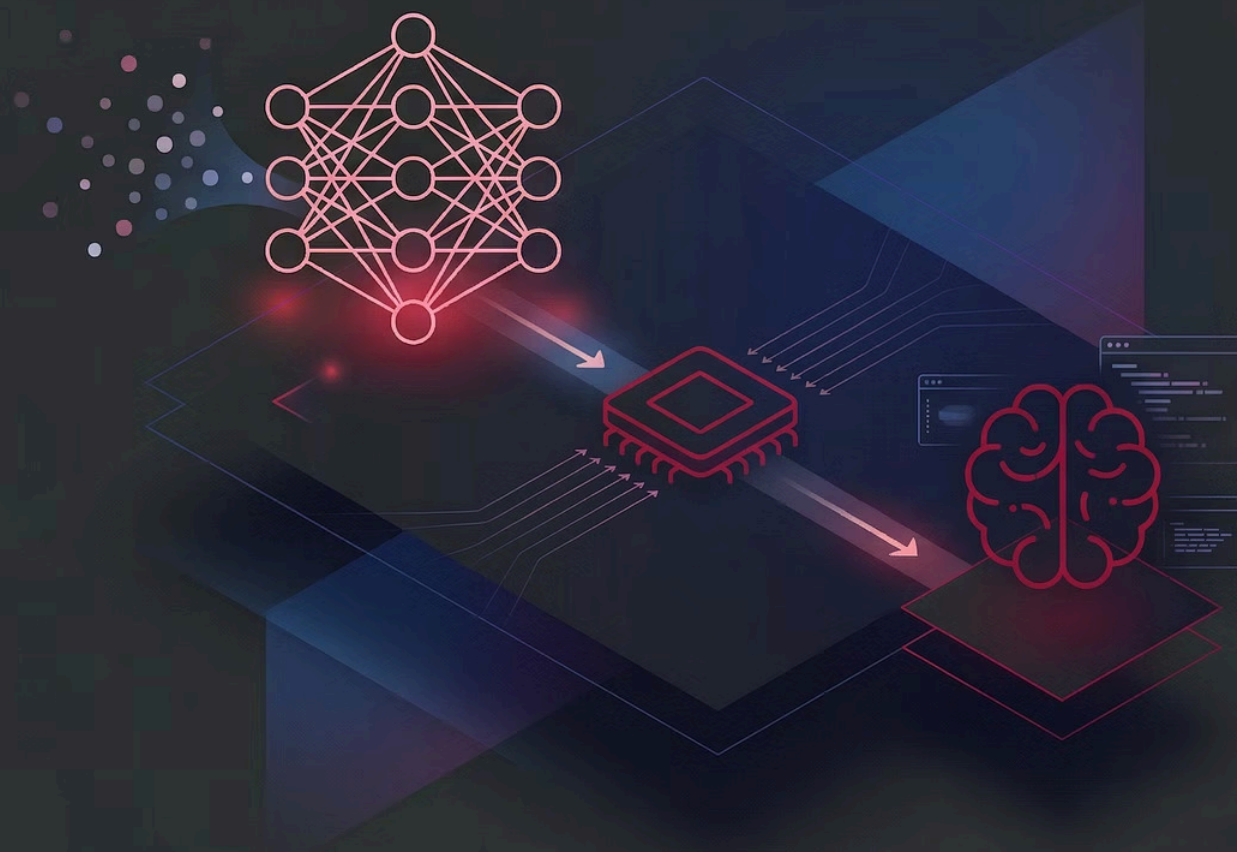
## 3 Deploy в Production

Масштабируйте и внедряйте решение в реальную среду



# Путь В: ML

1. Deep dive в архитектуру LLM
2. Fine-tuning модели на своём датасете
3. Создайте специализированный агент



# Путь С: Leadership

1. Веди code review в команде с фокусом на AI-код
2. Напишите гайдлайны для организации
3. Наставляй младших коллег



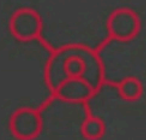
# Месяцы 5-6: Портфолио



1-2 серьёзных  
проекта в GitHub



Статья/блог пост  
о вайб-кодинге



Разговоры с людьми  
на целевых должностях



Участие в open source



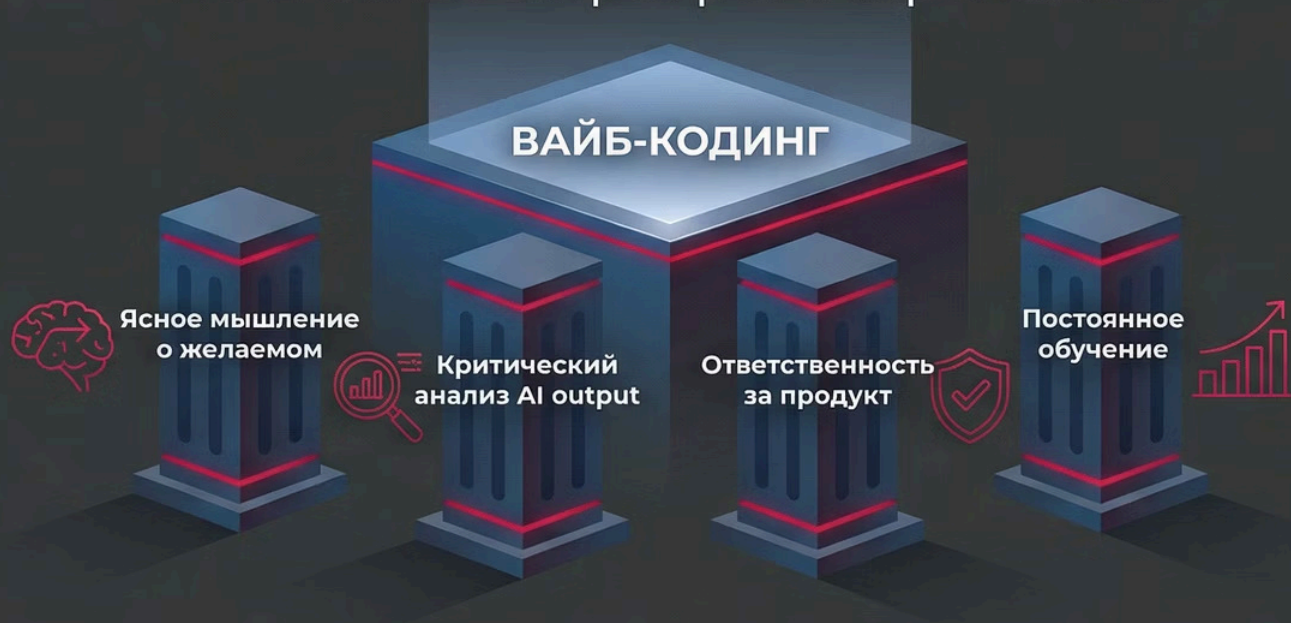


# 7. Философия вайб-кодинга

Интеграция творчества и искусственного интеллекта для создания гармоничного будущего.

# Вайб-кодинг — это новая дисциплина

Не ленивое программирование



# Вы стоите на пороге профессии, которая 5 лет назад не существовала

Инструменты будут меняться. Модели будут эволюционировать.



# Принципы, которые останутся



1. Автоматизируй то,  
что можно автоматизировать



2. Проверяй то, что нельзя  
автоматизировать



3. Думай о том, что  
и почему ты делаешь



4. Неси ответственность  
за результат

# Удачи в ваших проектах

Мир программирования ждёт вас

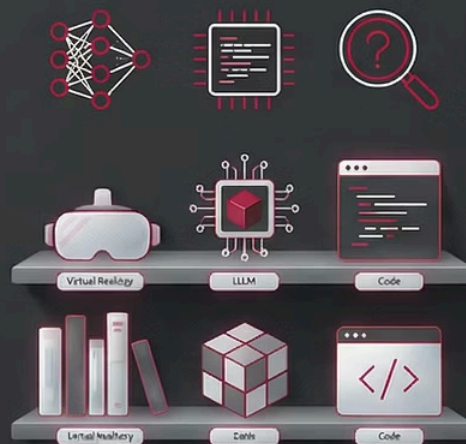


# Приложение: Ресурсы для дальнейшего обучения

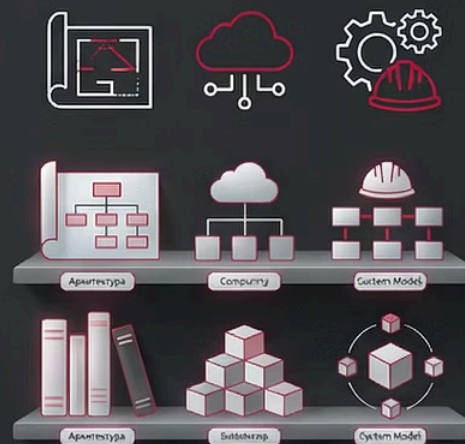
## Soft Skills



## AI/LLM



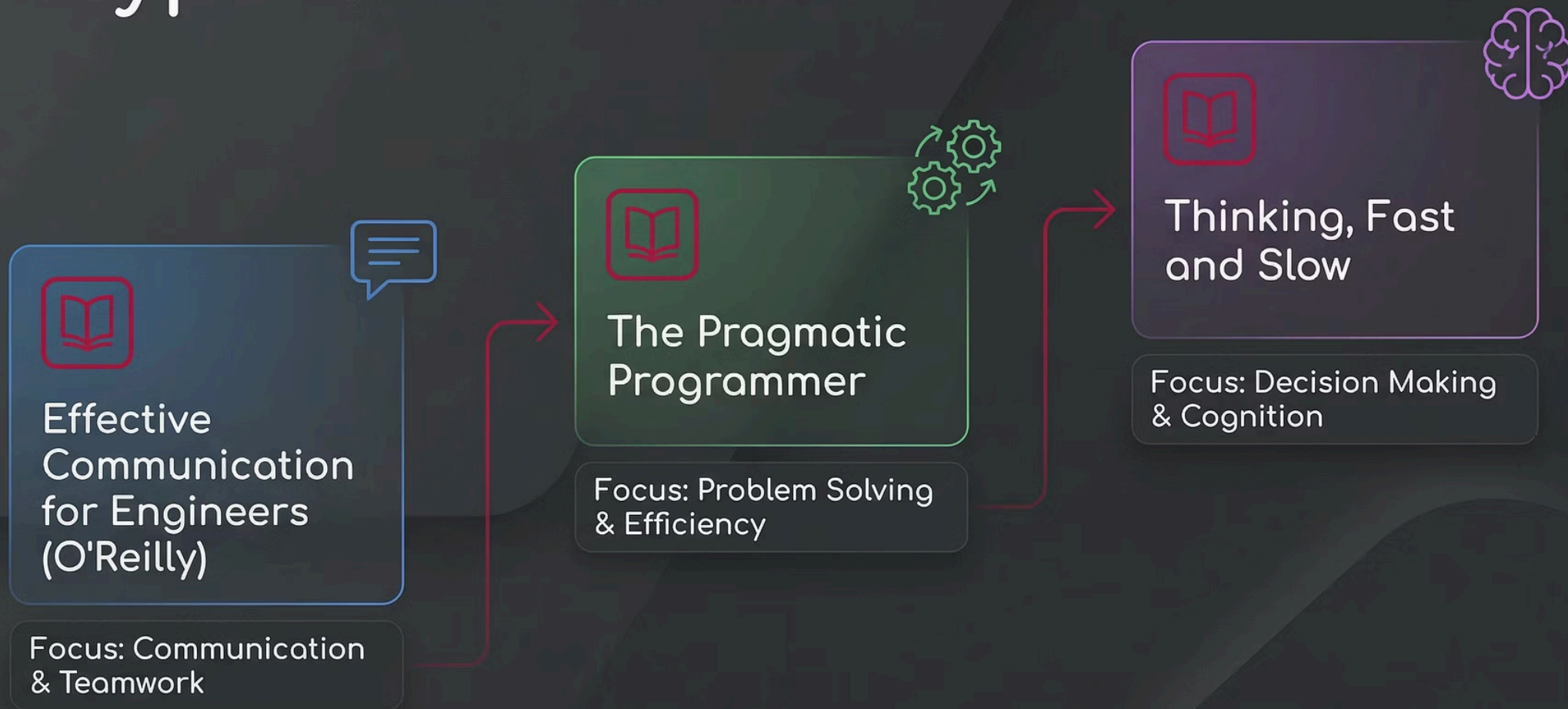
## Архитектура



## Community



# Ресурсы: Soft Skills



# Ресурсы: AI/LLM



## Курсы



DeepLearning.AI

DeepLearning.AI



## Papers

- Attention is All You Need

- LLaMA 2 Technical Paper



## Практика



HuggingFace

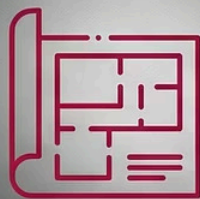
kaggle

Kaggle

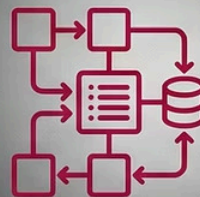


OpenRouter

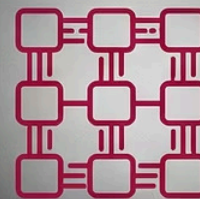
# Ресурсы: Архитектура



System Design  
Interview by  
Gaurav Sen



Building ML  
Powered  
Applications  
(Emmanuel  
Ameisen)



Architecture  
Patterns with  
Python

# Community: Где найти единомышленников

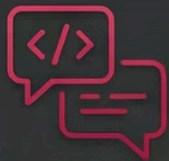


## Local tech meetups



Регулярные встречи разработчиков и энтузиастов в вашем городе. Обмен опытом, нетворкинг и живое общение.

## GitHub discussions



Обсуждение проектов, вопросов и идей непосредственно в репозиториях. Совместная работа над open-source проектами.

## Academic конференции: NeurIPS, ICML, CoNLL, SemEval



Международные научные мероприятия. Презентации исследований, воркшопы и общение с ведущими учеными.



# Ваше путешествие ТОЛЬКО начинается

Вайб-кодинг — это не пункт назначения,  
это способ мышления

